

Current Recursion and Tensor-Network Execution in PYAMPLICOL

Purpose. This note summarizes the matrix-element generation strategy used by AMPLICOL and the corresponding PYAMPLICOL implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AMPLICOL emits Fortran code that numerically sweeps this current table. PYAMPLICOL builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

Current status. The generic LC/NLC/full-colour staged-DAG implementation is the production PYAMPLICOL workflow. Process generation writes schema-v2 artifacts loaded by `rusticol`; model parameters are runtime inputs and may be overridden from TOML. The default benchmark route is Rusticol with SymJIT stage evaluators and stage-local inputs. Selected-flow JIT artifacts measure stage chunks around base 128 and run at batch 128; all-flow artifacts use uniform chunk 8192 and batch 64. Both use ten Horner iterations, Symbolica’s default CPE choice, and enlarged Horner/common-pair limits. The production matrix JIT row uses SymJIT O1 throughout. Repeated ten-second selected-flow checks have not established a consistent, material O3 runtime gain for any current matrix cell, so no O3 substitution is shown; dedicated Z-family scans still report O3/ASM/C++ variants. Selected JIT generation benchmarks effective chunk sizes 64, 96, 128, 192, 256, and unchunked for each stage, retaining a non-default candidate only above a five-percent measured gain. On AArch64, candidate scores count one shared native input pack and sum only evaluator and output-unpack work across chunks, matching Rusticol’s fused stage call. Large all-flow and backend-comparison artifacts stay uniformly chunked. The result matrices are generated as separate $\text{\TeX}$ inputs, and the benchmark writers refresh `pyAmpliCol.pdf` after each completed cell. Generic current and colour-sector generation caps are disabled by default; long benchmark refreshes rely on the 30 GB watchdog rather than truncated colour plans. On macOS the watchdog enforces the larger of process-tree RSS and compressed physical footprint, including descendants that create independent process sessions.

Once runnable stage evaluators are serialized, the saved schema compacts full generation-only interaction records to stage interaction IDs and vertex-kind counts. The full records remain available in `runtime_schema_diagnostics.pickle.gz`; plan-only manifests stay detailed, and Rusticol remains backward-compatible with earlier artifacts. On measured all-flow artifacts this removes about 80 percent of manifest bytes without changing any evaluator, runtime schedule, or matrix-element value.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}_{t,\chi}^\alpha(I) = P^\alpha{}_\beta(t, p_I) \sum_{\substack{I=A \cup B \\ A \cap B = \emptyset}} \mathcal{V}_{\gamma\delta}^\beta(t_A, t_B \rightarrow t) \mathcal{J}_{t_A, \chi_A}^\gamma(A) \mathcal{J}_{t_B, \chi_B}^\delta(B). \quad (1)$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i \in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AMPLICOL’s optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = (t, \chi, \text{bin}(I), \text{ext_cur}), \quad (2)$$

where `bin(I)` records the external subset and `ext_cur` is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AMPLICOL appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```

subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine

```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that PYAMPLICOL keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

PYAMPLICOL first builds an explicit Python representation of the same current table. In the generic compiler, a current is identified by a model-driven physics index rather than by a process-family name. The identity used for deduplication is

$$K_{\text{PYAMPLICOL}} = (\text{particle_id}, L, H, \chi, S, F, Q, C, P, A), \quad (3)$$

where L is the sorted tuple of external labels, H is the AMPLICOL-style helicity-source ancestry bitset, χ is chirality, S is any spin-state tag needed by the source or propagator, F is flavour flow, Q is charge flow, C is the leading-color state, P is the momentum mask, and A is an optional auxiliary kind such as the antisymmetric tensor used for the four-gluon vertex. The implementation also carries `ordered_external_labels`. That field is leading-color ordering metadata used by the color engine and amplitude grouping, but it is deliberately not part of equality or hashing. Deduplication is therefore exactly equality of Eq. (3); no rule is allowed to ask whether the process is “ Z plus jets” or any other family.

field	role in the current identity
<code>particle_id</code>	Model/UFO particle id, e.g. 21, 23, 1, -1 , or an auxiliary id.
<code>external_labels</code> and <code>external_mask</code>	Which external legs are contained in the current. The mask is the bit representation of the labels.
<code>helicity_ancestry</code>	Bitwise union of the source-current helicity bits feeding this current. This is how helicity configurations are woven into one shared DAG.
<code>chirality</code> and <code>spin_state</code>	Local spin information, especially for Weyl fermions and massive-vector sources.
<code>flavour_flow</code> and <code>charge_flow</code>	Quantum-number flow checked locally by model vertices, e.g. $u \rightarrow dW^+$ or $e^+e^- \rightarrow \gamma/Z$.
<code>color_state</code>	Colour-ordered open-line or trace state used to build the amplitude vector. LC, NLC and full colour differ in the final sparse colour contraction over this vector.
<code>momentum_mask</code>	Which all-outgoing momenta are summed for propagators and momentum-dependent vertices.
<code>auxiliary_kind</code>	Optional tag distinguishing auxiliary tensor currents from ordinary particles.

Source currents are built from external leg metadata. Internal currents are then generated by increasing current size, with all interactions that feed the same current index attached to one node.

```

for external_helicity_source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible_split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained_helicity_ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()

```

At runtime, PYAMPLICOL mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \text{current_rows}[s, i, c], \quad T_{s,v,c} \equiv \text{interaction_rows}[s, v, c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```

fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents] = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2 = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)

```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and hep_lib

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^\rho(p_A, p_B) A^\mu B^\nu, \quad \text{or in code: } V(m("a"), m("b"), m("out")) * A(m("a")) * B(m("b")).$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```
library = model.build_tensor_library()      # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())
```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spenso executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, PYAMPLICOL interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AMPLICOL's execution model.

The current NLC/full-colour implementation keeps the kinematic DAG colour-ordered and applies a sparse colour matrix to the amplitude vector. For open-line sectors, including arbitrary many quark lines with gluons distributed on the lines, PYAMPLICOL contracts the product of left colour strings with the conjugated right strings into closed fundamental traces. Those small colour trace expressions are simplified separately from the kinematic DAG; NLC is obtained by keeping the leading and first $1/N_c^2$ -suppressed powers, while full colour keeps the complete sparse metric. If a later implementation puts explicit colour tensors inside a Spenso block, colour simplification still belongs before Spenso execution:

```
raw ← idenso.simplify_color(raw),      then build TensorNetwork(raw, hep_lib).
```

For the present NLC/full-colour milestone, colour algebra is confined to the small colour-basis overlap problem and the Rusticol final contraction, not to expanded kinematic expressions. Fortran AmpliCol is the direct reference for zero, one and two quark-line colour matrices; beyond that, pyAmpliCol's generic open-line metric is validated through raw-amplitude probes and internal colour consistency checks.

4 Example: $d\bar{d} \rightarrow Zgg$

This section spells out one concrete DAG so that the split between compiled current contractions and Rust-interpreted scheduling is explicit. The user-level process is

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_3) g(p_4) g(p_5).$$

PYAMPLICOL stores the process in the all-outgoing convention. Thus labels $1, \dots, 5$ carry the fields

$$\bar{d}_1, \quad d_2, \quad Z_3, \quad g_4, \quad g_5,$$

with momenta $-p_1, -p_2, p_3, p_4$, and p_5 , respectively. A label set such as $\{2, 4, 5\}$ is represented by $L = (2, 4, 5)$ and by the mask

$$m(2, 4, 5) = 2^{2-1} + 2^{4-1} + 2^{5-1} = 26.$$

The momentum mask is the same bit pattern in this example, and it tells Rusticol which stored momentum sum must be passed to the next compiled evaluator.

For one leading-color sector, the colored ordered labels are

$$O = (2, 4, 5, 1),$$

with the color-singlet Z_3 allowed to attach anywhere on the quark line. The color engine may also provide the sector $O = (2, 5, 4, 1)$. Nothing in the DAG builder special-cases this as a “ $Z + 2g$ ” process: source currents are ordinary external-leg currents and all later currents are discovered from model vertices.

Source current indices

For a fixed retained source helicity choice, the source entries look as follows. The bit b_i denotes the unique helicity-source bit assigned to that external state.

source	current	mask	index payload
S_1	$\mathcal{J}_{\bar{d}}(\{1\})$	1	(pid = -1, $L = (1)$, $H = b_1, \chi, S, F = (-1), Q, C_{\bar{q}}, P = 1$)
S_2	$\mathcal{J}_d(\{2\})$	2	(pid = 1, $L = (2)$, $H = b_2, \chi, S, F = (1), Q, C_q, P = 2$)
S_3	$\epsilon_Z(\{3\})$	4	(pid = 23, $L = (3)$, $H = b_3, S = h_Z, C_{\text{singlet}}, P = 4$)
S_4	$\epsilon_g(\{4\})$	8	(pid = 21, $L = (4)$, $H = b_4, S = h_4, C_g, P = 8$)
S_5	$\epsilon_g(\{5\})$	16	(pid = 21, $L = (5)$, $H = b_5, S = h_5, C_g, P = 16$)

The actual stored index also contains the exact leading-color sector id, line-group metadata, and the current dimension. For this example the quark and anti-quark source currents have Weyl-spinor components, gluons and the Z have Lorentz-vector components, and auxiliary tensor currents have the antisymmetric tensor components required by the cubicized four-gluon rule.

What each compiled stage computes

The generic stage compiler groups interactions by the size of the result current. For each group it emits a multi-output Symbolica evaluator. That evaluator is compiled once and receives only parent-current components, momentum sums, and real couplings as parameters. It returns the components of all result currents in the stage.

For the sector $O = (2, 4, 5, 1)$, the schematic stage contents are:

$$E_2 : \quad \mathcal{J}_g(4, 5) = P_g(p_{45}) \mathcal{V}_{3g}(S_4, S_5; p_4, p_5), \quad (4)$$

$$\mathcal{J}_T(4, 5) = \mathcal{V}_{ggT}(S_4, S_5), \quad (5)$$

$$\mathcal{J}_d(2, 4) = P_q(p_{24}) \mathcal{V}_{ggq}(S_2, S_4; p_2, p_4), \quad (6)$$

$$\mathcal{J}_d(2, 3) = P_q(p_{23}) \mathcal{V}_{qZq}(S_2, S_3; p_2, p_3), \quad (7)$$

plus the analogous allowed currents for the other color sector and for the anti-quark side. The notation E_2 means a compiled evaluator for all two-label result currents, not a Python function that loops over these formulas component by component.

The next stage consumes the rows written by E_2 :

$$E_3 : \quad \mathcal{J}_d(2, 4, 5) = P_q(p_{245}) [\mathcal{V}_{ggq}(\mathcal{J}_d(2, 4), S_5) + \mathcal{V}_{ggq}(S_2, \mathcal{J}_g(4, 5))], \quad (8)$$

$$\mathcal{J}_d(2, 3, 4) = P_q(p_{234}) [\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4), S_3) + \mathcal{V}_{ggq}(\mathcal{J}_d(2, 3), S_4)]. \quad (9)$$

The stage compiler does not expand these expressions into diagrams. It collects all interaction records whose output index is the same current and compiles the sum into the output components of that current. If two different splits produce the same current index, their contributions are accumulated inside the same compiled stage output.

The four-label stage then builds the endpoint current used by the closure:

$$E_4 : \quad \mathcal{J}_d(2, 3, 4, 5) = P_q(p_{2345}) \sum_{\text{compatible splits}} \mathcal{V}(\mathcal{J}_{\text{left}}, \mathcal{J}_{\text{right}}). \quad (10)$$

The sum includes, for example, gluon-first and Z -insertion contributions such as

$$\mathcal{V}_{qZq}(\mathcal{J}_d(2, 4, 5), S_3), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3, 4), S_5), \quad \mathcal{V}_{qgq}(\mathcal{J}_d(2, 3), \mathcal{J}_g(4, 5)).$$

Those are still local vertex applications selected by the model and color engine. The result is one stored current row, identified by the full CurrentIndex and by its value-slot range in the artifact.

Finally, a compiled amplitude evaluator returns raw complex roots:

$$E_{\text{amp}} : \quad R_a = \mathcal{V}_{\text{close}}(\mathcal{J}_d(2, 3, 4, 5), S_1), \quad (11)$$

with one output for each retained root. Roots with the same $(H, \text{color sector}, O)$ coherent-group key are summed before squaring. The reported leading-color matrix element is

$$|\mathcal{M}|_{\text{LC}}^2 = \mathcal{N}_{\text{AC}} \sum_{\text{groups } g} w_g \left| \sum_{a \in g} R_a \right|^2. \quad (12)$$

The same example as a DAG

A stripped-down adjacency-list representation of this example is:

```
sources interpreted by Rusticol:
S1 = J(pid=-1, labels={1}, H=b1, color=qbar-line)
S2 = J(pid= 1, labels={2}, H=b2, color=q-line)
S3 = J(pid=23, labels={3}, H=b3, color=singlet)
S4 = J(pid=21, labels={4}, H=b4, color=line gluon)
S5 = J(pid=21, labels={5}, H=b5, color=line gluon)

compiled stage E2, subset_size=2:
I0: S4 + S5 -- V3g, Pg --> J(pid=21, labels={4,5})
I1: S4 + S5 -- VggT --> J(aux=T, labels={4,5})
I2: S2 + S4 -- Vqgq,Pq --> J(pid=1, labels={2,4})
I3: S2 + S3 -- VqZq,Pq --> J(pid=1, labels={2,3})

compiled stage E3, subset_size=3:
I4: J(2,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I5: S2 + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,4,5})
I6: J(2,4) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4})
I7: J(2,3) + S4 -- Vqgq,Pq --> J(pid=1, labels={2,3,4})

compiled stage E4, subset_size=4:
I8: J(2,4,5) + S3 -- VqZq,Pq --> J(pid=1, labels={2,3,4,5})
I9: J(2,3,4) + S5 -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})
I10: J(2,3) + J(4,5) -- Vqgq,Pq --> J(pid=1, labels={2,3,4,5})

compiled amplitude stage:
A0: J(pid=1, labels={2,3,4,5}) + S1 -- close --> R0
```

This listing is schematic: the real artifact contains all retained helicity ancestries, both leading-color sectors, source dimensions, output component ranges, momentum-slot ids, couplings, and color weights. The important point is the division of labor. The evaluator E_s performs the tensor contractions and the algebraic sum over interactions in stage s . Rusticol interprets the DAG schedule: fill sources, compute momentum sums, call E_2 , scatter its outputs into current storage, call E_3 , scatter again, and so on until E_{amp} and the final coherent square-sum.

5 Rusticol: the Implemented Rust Runtime

The previous staged PYAMPLICOL runtime proved that the shared-current DAG could match Fortran AMPLICOL numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation

keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/           # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` streams the typed schema directly from disk, validates current/value relationships through precomputed indices, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

On AArch64, the f64 route loads native two-lane complex SymJIT payloads. A stage-local input slice is packed once and reused by every output chunk in that stage; this keeps chunking as a code-size control without multiplying input-pack cost. Setting `RUSTICOL_NATIVE_SIMD_JIT=0` selects the scalar JIT fallback for diagnostics. Runnable artifacts likewise omit the full per-interaction diagnostic records after evaluator serialization, retaining compact IDs and per-kind counts. The trusted local diagnostic sidecar is a gzip-compressed Python protocol-5 pickle; it is not part of evaluator loading or execution.

```

for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;

```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica’s model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica’s `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema is process-generic. The staged execution format no longer needs a Rust-side process-family branch: each process directory supplies the source layout, current slots, stage records, amplitude roots, coherent groups, and evaluator artifacts needed by Rusticol. If a model vertex or color rule is not implemented yet, generation fails with an explicit unsupported-kernel diagnostic rather than falling back to a hidden family path.

6 Shared LC Ordering Recycling

LC process generation now has two related modes. For ordinary integration benchmarks, `PYAMPLICOL` times one selected LC ordering per phase-space point. The selected artifact is pruned before Symbolica evaluator construction, so no dynamic colour-flow selector remains in those expressions. For all-flow fixed-helicity matrix entries, `PYAMPLICOL` writes a materialized shared all-sector artifact: internal currents live in one shared LC sector, while amplitude roots retain the full colour-order mapping used for the `AMPLICOL imode=2` comparison. This avoids the old sector-local all-ordering clone while keeping the runtime result directly value-comparable.

The shared all-sector mode reuses pure-gluon subcurrents through the usual LC reflection relation, with the corresponding sign carried as an interaction colour weight. That sign is applied before current-stage propagation, so the shared DAG is numerically equivalent to the sector-specialized artifact while avoiding one current table per ordering. Runtime selection of one LC ordering is still available through generated sidecars; `time-process-lc-sector-ids` loads the matching pruned sidecar when it exists.

The fixed-helicity workload is selected without constructing an all-helicity all-sector artifact. The matrix driver derives replay-compatible candidates from the external source-spin states, evaluates their selected-sector raw sums at a deterministic point, and keeps the strongest nonzero candidate. Exact sector-id lookup is indexed, so this setup remains linear in the number of LC sectors rather than repeatedly scanning the complete colour plan.

Process	Orders	Curr.	AMPLICOL/PYAMPLICOL O1 gen	O1 JIT frac.	Sel. ratio	AMPLICOL/PYAMPLICOL all run	Rel. diff.
$gg \rightarrow t\bar{t} + g$	6	29	0.0967 / 0.172	0.328	1.76×	1.45 / 0.628	1.4×10^{-16}
$gg \rightarrow t\bar{t} + 2g$	24	118	0.105 / 0.267	0.587	1.42×	4.35 / 2.54	2.15×10^{-15}
$gg \rightarrow t\bar{t} + 3g$	120	592	0.13 / 1.22	0.772	1.16×	25.4 / 29.7	0
$gg \rightarrow t\bar{t} + 4g$	720	3554	0.412 / 10	0.826	1.11×	199 / 293	4.77×10^{-16}
$gg \rightarrow t\bar{t} + 5g$	5040	24880	5.91 / 123	0.874	1.19×	2.85×10^3 / 3.08×10^3	6.23×10^{-15}
$gg \rightarrow t\bar{t} + 6g$	40320	199042	272 / 1.26×10^3	0.862	1.53×	5.29×10^4 / 5.99×10^4	7.57×10^{-16}

The generation column reports `AMPLICOL/PYAMPLICOL` all-flow setup time in seconds for the fixed-helicity `imode=2`-style comparison. All run entries are wall microseconds per phase-space point, `AMPLICOL/PYAMPLICOL`. The selected-ratio column is the `PYAMPLICOL/AMPLICOL` multiplier for one LC flow with the usual helicity sum; the all-run column is the fixed-source-helicity sum over all LC flows. The generated table is refreshed from the LC matrix cache. At $gg \rightarrow t\bar{t} + 6g$, both implementations report 199,042 filtered currents and 40,320 amplitudes/orderings; the selected-flow validation agrees to better than 10^{-15} . The process-matrix tables below use the same selected- and all-flow runtime semantics.

High-multiplicity AArch64 limit. For $d\bar{d} \rightarrow t\bar{t} + 7g$, an all-flow O1 build hit SymJIT’s `x < 4096` assertion because its stack frame exceeded the shifted 12-bit ADD/SUB immediate. Patch 0003 splits the adjustment, and its regression and the matrix retry passed that point. The retry later reached `generic_stage_9_subset_10` but peaked at 30.118 GiB. The selected O1 artifact remains valid at 5.4×10^{-14} relative difference; the all-flow slot is marked >30 GB RAM, with partial outputs retained.

7 Generic LC Performance Matrix (SymJIT O1)

Cell format: generation time above runtime per phase-space point; slots are AMPLiCOL, PYAMPLiCOL SymJIT O1 selected flow, and PYAMPLiCOL SymJIT O1 all flows. AMPLiCOL uses generated-library timings. Conventions and gaps are summarized after the table.

ID	base process	n=1					n=2					n=3				
1	$d\bar{d} \rightarrow Z + (n-1)g$	1.873	/ 0.101	(x0.099)	(x1.34)		1.996	/ 0.102	(x0.113)	(x1.41)		1.958	/ 0.0944	(x0.154)	(x1.64)	
		0.0193	/ 0.859	(x6.87)	(x0.089)		0.115	/ 0.968	(x2.69)	(x0.149)		0.424	/ 1.144	(x1.71)	(x0.246)	
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	1.886	/ 0.0989	(x0.092)	(x1.41)		1.986	/ 0.102	(x0.103)	(x1.51)		1.938	/ 0.0973	(x0.13)	(x1.64)	
		0.0119	/ 0.863	(x9.12)	(x0.089)		0.0717	/ 0.9	(x3.02)	(x0.158)		0.232	/ 1.177	(x2.11)	(x0.24)	
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.016	/ 0.101	(x0.102)	(x1.50)		1.985	/ 0.0953	(x0.116)	(x1.63)	
							0.0679	/ 0.938	(x2.64)	(x0.135)		0.154	/ 1.076	(x2.17)	(x0.183)	
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					2.028	/ 0.103	(x0.094)	(x1.48)		1.914	/ 0.0981	(x0.108)	(x1.72)	
							0.0335	/ 1.019	(x3.21)	(x0.111)		0.0613	/ 0.957	(x3.07)	(x0.187)	
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.064	/ 0.102	(x0.136)	(x1.50)		2.026	/ 0.0976	(x0.23)	(x1.61)	
							0.165	/ 0.983	(x2.36)	(x0.144)		0.693	/ 1.049	(x1.46)	(x0.232)	
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A					2.058	/ 0.103	(x0.135)	(x1.52)		2.283	/ 0.0967	(x0.176)	(x1.77)	
							0.102	/ 1.069	(x3.89)	(x0.197)		0.646	/ 1.454	(x1.76)	(x0.432)	
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.025	/ 0.103	(x0.098)	(x1.48)		2.031	/ 0.0965	(x0.125)	(x1.76)	
							0.0513	/ 1.01	(x4.22)	(x0.178)		0.234	/ 1.24	(x2.12)	(x0.343)	
8	$gg \rightarrow gg + (n-2)g$	N/A					1.916	/ 0.0937	(x0.118)	(x1.69)		2.117	/ 0.104	(x0.265)	(x1.94)	
							0.158	/ 1.03	(x2.02)	(x0.25)		0.855	/ 2.394	(x1.31)	(x0.451)	
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.424	/ 0.0988	(x0.379)	(x1.58)	
												1.477	/ 1.249	(x1.42)	(x0.246)	
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
gen O1 one-flow, hel. sum		1.87	1.89	1.88	1.88		1.92	2.06	2.01	2.02		1.91	2.42	2.08	2.03	
		x0.092	x0.099	x0.096	x0.096	x0.096	x0.094	x0.136	x0.112	x0.108	x0.112	x0.108	x0.379	x0.187	x0.154	x0.192
gen O1 all-flows, one-hel		0.0989	0.101	0.1	0.1		0.0937	0.103	0.101	0.102		0.0944	0.104	0.0976	0.0973	
		x1.34	x1.41	x1.37	x1.37	x1.37	x1.41	x1.69	x1.51	x1.50	x1.51	x1.58	x1.94	x1.70	x1.64	x1.70
run O1 one-flow, hel. sum		0.0119	0.0193	0.0156	0.0156		0.0335	0.165	0.0955	0.0866		0.0613	1.48	0.531	0.424	
		x6.87	x9.12	x8.00	x8.00	x7.73	x2.02	x4.22	x3.01	x2.85	x2.79	x1.31	x3.07	x1.90	x1.76	x1.59
run O1 all-flows, one-hel		0.859	0.863	0.861	0.861		0.9	1.07	0.99	0.996		0.957	2.39	1.3	1.18	
		x0.089	x0.089	x0.089	x0.089	x0.089	x0.111	x0.25	x0.165	x0.154	x0.166	x0.183	x0.451	x0.284	x0.246	x0.309

Generic LC Performance Matrix (SymJIT O1) (continued)

ID	base process	n=4				n=5				n=6			
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.056	/ 0.0961	(x0.369)	(x1.79)	2.32	/ 0.109	(x0.795)	(x2.54)	3.186	/ 0.213	(x1.28)	(x5.82)
		1.371	/ 1.954	(x1.36)	(x0.387)	4.356	/ 7.354	(x1.16)	(x0.39)	13.93	/ 41.84	(x0.961)	(x0.75)
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.003	/ 0.097	(x0.247)	(x1.79)	2.138	/ 0.108	(x0.57)	(x2.58)	2.495	/ 0.22	(x1.18)	(x5.62)
		0.864	/ 2.048	(x1.46)	(x0.372)	2.801	/ 7.252	(x1.27)	(x0.395)	9.301	/ 43.76	(x1.02)	(x0.69)
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.055	/ 0.097	(x0.174)	(x1.68)	2.196	/ 0.106	(x0.264)	(x1.68)	2.493	/ 0.172	(x0.57)	(x1.68)
		0.44	/ 1.228	(x1.60)	(x0.285)	1.32	/ 2.24	(x1.28)	(x0.388)	3.995	/ 8.14	(x1.12)	(x0.389)
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2	/ 0.101	(x0.146)	(x1.54)	2.119	/ 0.126	(x0.19)	(x1.41)	2.208	/ 0.157	(x0.425)	(x1.80)
		0.155	/ 1.178	(x2.47)	(x0.286)	0.645	/ 2.173	(x1.53)	(x0.387)	2.123	/ 7.369	(x1.31)	(x0.411)
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	2.319	/ 0.103	(x0.441)	(x1.58)	2.578	/ 0.115	(x0.905)	(x1.71)	3.521	/ 0.19	(x1.29)	(x2.04)
		2.073	/ 1.557	(x1.23)	(x0.313)	6.281	/ 3.411	(x0.989)	(x0.377)	17.1	/ 13.52	(x0.937)	(x0.346)
6	$gg \rightarrow t\bar{t} + (n-2)g$	3.189	/ 0.105	(x0.369)	(x2.54)	6.277	/ 0.13	(x0.471)	(x9.39)	25.49	/ 0.412	(x0.241)	(x24.3)
		2.352	/ 4.349	(x1.42)	(x0.584)	7.8	/ 25.36	(x1.16)	(x1.17)	22.51	/ 199	(x1.11)	(x1.47)
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	2.274	/ 0.104	(x0.229)	(x2.10)	2.971	/ 0.124	(x0.439)	(x4.39)	4.123	/ 0.233	(x0.78)	(x13.9)
		0.864	/ 2.456	(x1.54)	(x0.525)	2.839	/ 10.48	(x1.35)	(x0.507)	8.926	/ 65.35	(x1.11)	(x1.01)
8	$gg \rightarrow gg + (n-2)g$	2.182	/ 0.11	(x0.597)	(x5.55)	2.829	/ 0.198	(x1.08)	(x23.9)	5.012	/ 3.471	(x1.36)	(x15.2)
		3.178	/ 13.22	(x1.22)	(x0.964)	10.32	/ 105	(x1.03)	(x1.13)	32.19	/ 977	(x0.966)	(x1.51)
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	2.97	/ 0.102	(x0.685)	(x1.53)	4.709	/ 0.124	(x0.798)	(x1.41)	8.469	/ 0.208	(x0.864)	(x1.21)
		4.665	/ 1.548	(x1.13)	(x0.319)	13	/ 2.504	(x0.91)	(x0.38)	33.08	/ 6.732	(x0.959)	(x0.368)
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	3.694	/ 0.0989	(x0.192)	(x1.66)	4.471	/ 0.115	(x0.211)	(x1.48)	5.312	/ 0.166	(x0.289)	(x1.12)
		0.742	/ 1.459	(x1.99)	(x0.334)	1.581	/ 1.662	(x1.67)	(x0.422)	3.787	/ 2.579	(x1.27)	(x0.447)
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	2.945	/ 0.103	(x0.218)	(x1.68)	4.94	/ 0.135	(x0.31)	(x1.77)	10.1	/ 0.238	(x0.398)	(x2.30)
		1.043	/ 1.547	(x1.58)	(x0.492)	3.329	/ 2.811	(x1.36)	(x0.624)	11.48	/ 8.833	(x1.11)	(x0.605)
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	3.879	/ 0.106	(x0.145)	(x1.55)	4.611	/ 0.117	(x0.2)	(x1.39)	5.585	/ 0.167	(x0.271)	(x1.06)
		1.394	/ 1.691	(x0.999)	(x0.238)	2.763	/ 2.026	(x0.913)	(x0.278)	6.406	/ 2.974	(x0.71)	(x0.302)
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	2.188		(x0.107)	0.234	3.18		(x0.102)	0.324	5.499		(x0.129)	0.708
		0.173		(x1.95)	(N/A)	0.662		(x1.18)	(N/A)	2.289		(x0.845)	(N/A)
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A		2.149	2.149
										N/A		(5.62)	(N/A)
gen O1 one-flow, hel. sum		2	3.88	2.6	2.27	2.12	6.28	3.49	2.97	2.21	25.5	6.42	5.01
		x0.107	x0.685	x0.301	x0.229	x0.102	x1.08	x0.487	x0.439	x0.129	x1.36	x0.698	x0.57
gen O1 all-flows, one-hel		0.0961	0.11	0.102	0.102	0.106	0.198	0.126	0.121	0.157	3.47	0.487	0.21
		x1.53	x5.55	x2.08	x1.68	x1.39	x23.9	x4.47	x1.74	x1.06	x24.3	x6.34	x2.17
run O1 one-flow, hel. sum		0.155	4.67	1.49	1.04	0.645	13	4.44	2.84	2.12	33.1	12.9	9.3
		x0.999	x2.47	x1.53	x1.46	x0.91	x1.67	x1.22	x1.18	x0.71	x1.31	x1.03	x1.02
run O1 all-flows, one-hel		1.18	13.2	2.85	1.62	1.66	105	14.4	3.11	2.58	977	115	11.2
		x0.238	x0.964	x0.425	x0.353	x0.278	x1.17	x0.538	x0.393	x0.302	x1.51	x0.691	x0.526
					x0.626				x0.967				x1.39

Generic LC Performance Matrix (SymJIT O1) (continued)

ID	base process	n=7					n=8					n=9				
1	$d\bar{d} \rightarrow Z + (n-1)g$	4.109	/ 1.167	(x1.96)	(x8.17)		8.81	/ 16.39	(x1.89)	(x7.45)		24.92	/ 517	(x1.59)	(x2.70)	
		34.92	/ 287	(x1.07)	(x1.06)		90.29	/ 5e+03	(x1.08)	(x0.607)		226	/ 8.89e+04	(x1.32)	(x0.791)	
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	3.031	/ 1.164	(x2.15)	(x8.42)		4.618	/ 16.03	(x2.89)	(x7.63)		10.29	/ 491	(x2.99)	(x2.81)	
		25.54	/ 297	(x1.05)	(x1.03)		68	/ 5.34e+03	(x1.09)	(x0.594)		170	/ 8.44e+04	(x1.40)	(x0.675)	
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.67	/ 0.663	(x1.33)	(x1.95)		3.701	/ 6.831	(x2.07)	(x1.47)		7.045	/ 91.97	(x2.21)	(x1.01)	
		11.65	/ 45.42	(x0.973)	(x0.645)		32.19	/ 322	(x1.01)	(x0.966)		84.17	/ 6.47e+03	(x1.14)	(x0.567)	
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.282	/ 0.653	(x0.968)	(x1.91)		2.655	/ 6.764	(x1.98)	(x1.43)		6.338	/ 101	(x5.82)	(x2.34)	
		6.815	/ 42.52	(x1.15)	(x0.673)		20.38	/ 302	(x1.05)	(x1.03)		59.59	/ 6.74e+03	(x1.26)	(x0.726)	
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	6.452	/ 0.773	(x1.32)	(x2.51)		16.52	/ 7.949	(x1.08)	(x1.97)		27.85	/ 107	(x1.44)	(x1.13)	
		43.37	/ 77.24	(x0.956)	(x0.699)		106	/ 539	(x0.968)	(x0.925)		251	/ 1.26e+04	(x0.971)	(x0.439)	
6	$gg \rightarrow t\bar{t} + (n-2)g$	221	/ 5.905	(x0.059)	(x20.8)		2.3e+03	/ 272	(x0.013)	(x4.63)		N/A				
		59.82	/ 2.85e+03	(x1.19)	(x1.08)		153	/ 5.29e+04	(x1.53)	(x1.13)						
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	10.68	/ 1.369	(x0.638)	(x23.4)		70.48	/ 25.68	(x0.219)	(x13.5)		605	/ 1.21e+03	(x0.054)	>30 GB RAM	
		25.31	/ 471	(x1.12)	(x1.32)		70.57	/ 9.94e+03	(x1.09)	(x0.81)		170	/ 1.22e+05	(x1.49)	>30 GB RAM	
8	$gg \rightarrow gg + (n-2)g$	10.5	/ 232	(x1.52)	(x7.55)		51.71	/ 2.48e+04	(x0.799)	>30 GB RAM		N/A				
		82.77	/ 2.62e+04	(x1.25)	(x1.10)		209	/ 3.81e+05	(x1.32)	>30 GB RAM						
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	22.21	/ 0.867	(x0.644)	(x0.744)		66.46	/ 6.915	(x0.481)	(x0.524)		149	/ 80.59	(x0.529)	(x0.372)	
		80.67	/ 28.11	(x0.954)	(x0.347)		186	/ 153	(x0.922)	(x0.721)		454	/ 1.4e+03	(x0.86)	(x0.733)	
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	7.707	/ 0.711	(x0.4)	(x0.351)		14.08	/ 6.051	(x0.467)	(x0.097)		31.49	/ 72.86	(x0.375)	(x0.042)	
		9.804	/ 6.02	(x1.06)	(x0.417)		24.15	/ 24.49	(x0.986)	(x0.329)		60.16	/ 132	(x1.08)	(x0.823)	
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	31.04	/ 0.989	(x0.291)	(x2.39)		134	/ 7.424	(x0.156)	(x2.17)		1.33e+03	/ 81.88	(x0.036)	(x1.05)	
		31.54	/ 44.41	(x1.21)	(x1.07)		85.72	/ 263	(x1.22)	(x1.27)		219	/ 2.98e+03	(x1.06)	(x0.904)	
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	8.688	/ 0.719	(x0.318)	(x0.31)		16.2	/ 6.069	(x0.352)	(x0.076)		37.02	/ 69.15	(x0.301)	(x0.045)	
		16.34	/ 7.102	(x0.572)	(x0.266)		36.43	/ 27.39	(x0.577)	(x0.223)		86.55	/ 145	(x0.611)	(x0.723)	
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	13.03		(x0.128)	1.661		35.07		(x0.117)	4.094		N/A				
		7.384		(x0.682)	(N/A)		21.44		(x0.621)	(N/A)						
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A		11.25	11.25		N/A		>30 GB RAM	>30 GB RAM		N/A				
		N/A		(65.4)	N/A		N/A		>30 GB RAM	>30 GB RAM						
gen O1 one-flow, hel. sum		2.28	221	26.4	8.69		2.66	2.3e3	209	16.5		6.34	1.33e3	223	29.7	
		x0.059	x2.15	x0.902	x0.644	x0.278	x0.013	x2.89	x0.964	x0.481	x0.08	x0.036	x5.82	x1.53	x0.983	x0.155
gen O1 all-flows, one-hel		0.653	232	20.5	0.928		6.05	2.48e4	2.1e3	7.69		69.1	1.21e3	283	96.5	
		x0.31	x23.4	x6.54	x2.45	x7.84	x0.002	x13.5	x3.41	x1.72	x0.077	x0.027	x2.81	x1.15	x1.03	x1.20
run O1 one-flow, hel. sum		6.82	82.8	33.5	25.5		20.4	209	84.8	70.6		59.6	454	178	170	
		x0.572	x1.25	x1.02	x1.06	x1.07	x0.577	x1.53	x1.04	x1.05	x1.13	x0.611	x1.49	x1.12	x1.11	x1.09
run O1 all-flows, one-hel		6.02	2.62e4	2.53e3	61.3		24.5	3.81e5	3.8e4	431		132	1.22e5	3.26e4	6.6e3	
		x0.266	x1.32	x0.809	x0.867	x1.09	x0.223	x1.27	x0.783	x0.81	x1.01	x0.439	x0.904	x0.709	x0.726	x0.713

Each AMPLICOL pair is selected-flow/helicity-summed then all-flow/fixed-helicity; PYAMPLICOL SymJIT O1 uses matching pruned and shared artifacts. Entries are generation seconds above runtime wall microseconds per point; PYAMPLICOL-only cells are absolute. Green/orange/red means below one/below two/at least two. Summaries are min|max|avg|med|sum; sum compares paired totals. >30 GB RAM marks only the affected selected/all-flow workload. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8} .

Settings. docs/result_matrix.py (-color-accuracy lc) writes this table and its JSON cache. AMPLICOL uses its generated library for the selected workload and the fixed-helicity imode=2 colour probe for all flows. PYAMPLICOL writes separate selected_flow and shared all_flows artifacts. PYAMPLICOL uses Rusticol double precision; selected/all-flow batches are 128/64. Selected JIT stages measure chunks around base 128; all-flow outputs use uniform chunk 8192. Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and -target-runtime 10 are used. The two LC workloads are displayed side by side.

Runtime interpretation. On AArch64, Rusticol runs saved SymJIT stages as native two-lane complex evaluators and packs each stage input once across its chunks. Wall time measures the public evaluate call; the profiler separates evaluator and packing work. Low-multiplicity fixed overhead can dominate an AMPLICOL call measured in nanoseconds. Ratios at or above two are reviewed throughout, with $n \geq 5$ as the scaling gate.

8 Dedicated $d\bar{d} \rightarrow Z + \text{Gluon}$ Performance

n process	route	setup	selected flow, helicity sum				all flows, fixed helicity				notes
			gen [s]	wall [us/pt]	eval [us/pt]		gen [s]	wall [us/pt]	eval [us/pt]		
1 d d ⁺ > Z + (1-1)*g	reference	AMPLICOL	1.87	N/A	0.0193		0.101	N/A		0.859	
1 d d ⁺ > Z + (1-1)*g	Rusticol	PYAMPLICOL JIT O1	0.177 (x0.09)	0.137 (x7.11)	0.0347 (x1.80)		0.155 (x1.53)	0.0787 (x0.09)	0.0162 (x0.02)		
1 d d ⁺ > Z + (1-1)*g	Rusticol	PYAMPLICOL ASM O3	0.88 (x0.47)	0.185 (x9.57)	0.09 (x4.67)		0.672 (x6.65)	0.0957 (x0.11)	0.0314 (x0.04)		
1 d d ⁺ > Z + (1-1)*g	Rusticol	PYAMPLICOL C++ O3	0.767 (x0.41)	0.136 (x7.04)	0.0404 (x2.09)		0.722 (x7.14)	0.0892 (x0.10)	0.0246 (x0.03)		
1 d d ⁺ > Z + (1-1)*g	Rusticol	PYAMPLICOL JIT O3	0.179 (x0.10)	0.133 (x6.89)	0.0335 (x1.74)		0.154 (x1.52)	0.0778 (x0.09)	0.0159 (x0.02)		
2 d d ⁺ > Z + (2-1)*g	reference	AMPLICOL	2	N/A	0.115		0.102	N/A		0.968	
2 d d ⁺ > Z + (2-1)*g	Rusticol	PYAMPLICOL JIT O1	0.264 (x0.13)	0.323 (x2.81)	0.156 (x1.36)		0.164 (x1.61)	0.147 (x0.15)	0.0499 (x0.05)		
2 d d ⁺ > Z + (2-1)*g	Rusticol	PYAMPLICOL ASM O3	1.03 (x0.51)	0.698 (x6.06)	0.542 (x4.70)		1.02 (x10)	0.241 (x0.25)	0.14 (x0.14)		
2 d d ⁺ > Z + (2-1)*g	Rusticol	PYAMPLICOL C++ O3	1.26 (x0.63)	0.378 (x3.28)	0.224 (x1.95)		0.992 (x9.75)	0.164 (x0.17)	0.0674 (x0.07)		
2 d d ⁺ > Z + (2-1)*g	Rusticol	PYAMPLICOL JIT O3	0.199 (x0.10)	0.327 (x2.84)	0.155 (x1.35)		0.173 (x1.70)	0.143 (x0.15)	0.0492 (x0.05)		
3 d d ⁺ > Z + (3-1)*g	reference	AMPLICOL	1.96	N/A	0.424		0.0944	N/A		1.14	
3 d d ⁺ > Z + (3-1)*g	Rusticol	PYAMPLICOL JIT O1	0.233 (x0.12)	0.726 (x1.71)	0.439 (x1.04)		0.172 (x1.83)	0.287 (x0.25)	0.15 (x0.13)		
3 d d ⁺ > Z + (3-1)*g	Rusticol	PYAMPLICOL ASM O3	1.38 (x0.70)	1.87 (x4.42)	1.65 (x3.90)		1.23 (x13)	0.641 (x0.56)	0.507 (x0.44)		
3 d d ⁺ > Z + (3-1)*g	Rusticol	PYAMPLICOL C++ O3	2.29 (x1.17)	0.936 (x2.21)	0.69 (x1.63)		1.38 (x14.6)	0.335 (x0.29)	0.205 (x0.18)		
3 d d ⁺ > Z + (3-1)*g	Rusticol	PYAMPLICOL JIT O3	0.224 (x0.11)	0.751 (x1.77)	0.454 (x1.07)		0.173 (x1.83)	0.289 (x0.25)	0.147 (x0.13)		
4 d d ⁺ > Z + (4-1)*g	reference	AMPLICOL	2.06	N/A	1.37		0.0961	N/A		1.95	
4 d d ⁺ > Z + (4-1)*g	Rusticol	PYAMPLICOL JIT O1	0.367 (x0.18)	1.9 (x1.39)	1.32 (x0.96)		0.266 (x2.77)	0.766 (x0.39)	0.496 (x0.25)		
4 d d ⁺ > Z + (4-1)*g	Rusticol	PYAMPLICOL ASM O3	1.8 (x0.87)	5.38 (x3.92)	4.98 (x3.63)		1.55 (x16.2)	2.18 (x1.11)	1.95 (x1.00)		
4 d d ⁺ > Z + (4-1)*g	Rusticol	PYAMPLICOL C++ O3	5.48 (x2.67)	2.57 (x1.88)	2.2 (x1.61)		2.35 (x24.4)	0.981 (x0.50)	0.761 (x0.39)		
4 d d ⁺ > Z + (4-1)*g	Rusticol	PYAMPLICOL JIT O3	0.406 (x0.20)	1.96 (x1.43)	1.39 (x1.01)		0.192 (x2.00)	0.779 (x0.40)	0.505 (x0.26)		
5 d d ⁺ > Z + (5-1)*g	reference	AMPLICOL	2.32	N/A	4.36		0.109	N/A		7.35	
5 d d ⁺ > Z + (5-1)*g	Rusticol	PYAMPLICOL JIT O1	0.623 (x0.27)	5.54 (x1.27)	4.28 (x0.98)		0.304 (x2.79)	2.9 (x0.39)	2.16 (x0.29)		
5 d d ⁺ > Z + (5-1)*g	Rusticol	PYAMPLICOL ASM O3	3.88 (x1.67)	16.2 (x3.72)	15 (x3.44)		2.06 (x18.9)	9.52 (x1.29)	9.1 (x1.24)		
5 d d ⁺ > Z + (5-1)*g	Rusticol	PYAMPLICOL C++ O3	16.6 (x7.15)	8.07 (x1.85)	7.08 (x1.63)		8.38 (x76.9)	4.31 (x0.59)	3.8 (x0.52)		
5 d d ⁺ > Z + (5-1)*g	Rusticol	PYAMPLICOL JIT O3	0.581 (x0.25)	5.62 (x1.29)	4.37 (x1.00)		0.307 (x2.82)	2.94 (x0.40)	2.19 (x0.30)		
6 d d ⁺ > Z + (6-1)*g	reference	AMPLICOL	3.19	N/A	13.9		0.213	N/A		41.8	
6 d d ⁺ > Z + (6-1)*g	Rusticol	PYAMPLICOL JIT O1	1.14 (x0.36)	13.7 (x0.99)	11.5 (x0.83)		1.34 (x6.26)	32.9 (x0.79)	29 (x0.69)		
6 d d ⁺ > Z + (6-1)*g	Rusticol	PYAMPLICOL ASM O3	7.37 (x2.31)	46.5 (x3.33)	45 (x3.23)		4.49 (x21)	68.6 (x1.64)	66 (x1.58)		
6 d d ⁺ > Z + (6-1)*g	Rusticol	PYAMPLICOL C++ O3	46.9 (x14.7)	25.7 (x1.85)	24.3 (x1.75)		71.1 (x333)	57.1 (x1.36)	54.5 (x1.30)		
6 d d ⁺ > Z + (6-1)*g	Rusticol	PYAMPLICOL JIT O3	1.18 (x0.37)	15.5 (x1.11)	13.3 (x0.96)		1.37 (x6.40)	34.6 (x0.83)	31.4 (x0.75)		
7 d d ⁺ > Z + (7-1)*g	reference	AMPLICOL	4.11	N/A	34.9		1.17	N/A		287	
7 d d ⁺ > Z + (7-1)*g	Rusticol	PYAMPLICOL JIT O1	2.86 (x0.70)	47.6 (x1.36)	42.9 (x1.23)		10.1 (x8.63)	329 (x1.15)	306 (x1.07)		
7 d d ⁺ > Z + (7-1)*g	Rusticol	PYAMPLICOL ASM O3	14.8 (x3.61)	126 (x3.62)	121 (x3.47)		21.3 (x18.3)	583 (x2.03)	565 (x1.97)		
7 d d ⁺ > Z + (7-1)*g	Rusticol	PYAMPLICOL C++ O3	124 (x30.3)	77.4 (x2.22)	73.7 (x2.11)		t/o >15 min	N/A	N/A		
7 d d ⁺ > Z + (7-1)*g	Rusticol	PYAMPLICOL JIT O3	2.93 (x0.71)	51.8 (x1.48)	45.5 (x1.30)		10 (x8.58)	338 (x1.18)	321 (x1.12)		
8 d d ⁺ > Z + (8-1)*g	reference	AMPLICOL	8.81	N/A	90.3		16.4	N/A		5e3	
8 d d ⁺ > Z + (8-1)*g	Rusticol	PYAMPLICOL JIT O1	6.2 (x0.70)	138 (x1.53)	128 (x1.41)		85.6 (x5.22)	3.91e3 (x0.78)	3.76e3 (x0.75)		
8 d d ⁺ > Z + (8-1)*g	Rusticol	PYAMPLICOL ASM O3	32.5 (x3.69)	323 (x3.57)	324 (x3.58)		158 (x9.65)	6.58e3 (x1.31)	6.37e3 (x1.27)		
8 d d ⁺ > Z + (8-1)*g	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A		N/A	N/A	N/A		
8 d d ⁺ > Z + (8-1)*g	Rusticol	PYAMPLICOL JIT O3	6.28 (x0.71)	142 (x1.57)	131 (x1.45)		85.9 (x5.24)	3.92e3 (x0.78)	3.77e3 (x0.75)		
9 d d ⁺ > Z + (9-1)*g	reference	AMPLICOL	24.9	N/A	226		517	N/A		8.89e4	
9 d d ⁺ > Z + (9-1)*g	Rusticol	PYAMPLICOL JIT O1	13.7 (x0.55)	330 (x1.46)	305 (x1.35)		1.4e3 (x2.70)	6.34e4 (x0.71)	6.29e4 (x0.71)		
9 d d ⁺ > Z + (9-1)*g	Rusticol	PYAMPLICOL ASM O3	70 (x2.81)	858 (x3.79)	843 (x3.72)		1.98e3 (x3.83)	1.21e5 (x1.36)	1.26e5 (x1.41)		
9 d d ⁺ > Z + (9-1)*g	Rusticol	PYAMPLICOL C++ O3	N/A	N/A	N/A		N/A	N/A	N/A		
9 d d ⁺ > Z + (9-1)*g	Rusticol	PYAMPLICOL JIT O3	13.9 (x0.56)	394 (x1.74)	351 (x1.55)		1.4e3 (x2.72)	6.61e4 (x0.74)	7.05e4 (x0.79)		

Reproduce the $n = 7$ JIT O1 selected-flow entry with the following generation and timing commands.

```

env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol generate-process 'd d+ > z g g g g g' docs/.z_performance_outputs/manual/n7/jit_o1 --replace --n_cores 5 --color-accuracy 1c --batch-size 64 --symbolica-n-cores 5
--symbolica-output-chunk-size 128 --symbolica-output-chunk-strategy uniform --symbolica-stage-local-parameter-layout --symbolica-iterations 10 --symbolica-max-horner-scheme-variables 1000 --symbolica-max-common-pair-cache-entries 5000000
--symbolica-max-common-pair-distance 1000 --lc-sector-ids 0 --reference-color-order 2,4,5,6,7,8,9,1,3 --symbolica-evaluator-backend jit --symbolica-jit-optimization-level 1
env PYTHONPATH=src dependencies/.venv/bin/python -m pyamplicol time-process --target-runtime 10 --batch-size 64 --json docs/.z_performance_outputs/manual/n7/jit_o1

```

Here n is final-state multiplicity. Each block reports generation seconds and wall/evaluator microseconds per point: one selected flow with the helicity sum on the left, all flows at one fixed helicity on the right. Wall measures Rusticol's public `evaluate` call; eval separately profiles evaluator plus input packing. Both use batch 64, stage-local inputs, ten Horner iterations, and a ten-second timing target; output chunks are 128 selected and 8192 all-flow. The green O1 row is the measured default: repeated O3 timings showed no significant Z-family gain. ASM remains a scalar AArch64 diagnostic. The driver keeps generated artifacts under `docs/.z_performance_outputs`.

9 Generic NLC Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. AMPLICOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1					n=2					n=3				
1	$d\bar{d} \rightarrow Z + (n-1)g$	1.913	(x0.069)	(x0.492)			2.052	(x0.076)	(x0.627)			2.003	(x0.104)	(x1.73)		
		11.02	(x0.017)	(x0.005)	(x0.017)	(x0.005)	25.42	(x0.015)	(x0.008)	(x0.016)	(x0.009)	88.93	(x0.019)	(x0.012)	(x0.021)	(x0.016)
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	1.874	(x0.069)	(x0.402)			2.018	(x0.077)	(x0.563)			1.974	(x0.088)	(x1.19)		
		10.06	(x0.016)	(x0.004)	(x0.017)	(x0.004)	24.23	(x0.012)	(x0.006)	(x0.012)	(x0.006)	70.51	(x0.014)	(x0.009)	(x0.017)	(x0.012)
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.05	(x0.078)	(x0.563)			2.001	(x0.078)	(x0.837)		
							16.91	(x0.015)	(x0.007)	(x0.016)	(x0.007)	37.75	(x0.011)	(x0.006)	(x0.013)	(x0.008)
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					2.025	(x0.073)	(x0.52)			1.987	(x0.075)	(x0.716)		
							15.16	(x0.013)	(x0.005)	(x0.012)	(x0.004)	31.1	(x0.009)	(x0.004)	(x0.009)	(x0.004)
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.084	(x0.076)	(x0.653)			2.059	(x0.092)	(x1.44)		
							41.38	(x0.011)	(x0.006)	(x0.012)	(x0.007)	115	(x0.01)	(x0.006)	(x0.012)	(x0.009)
6	$g\bar{g} \rightarrow t\bar{t} + (n-2)g$	N/A					2.058	(x0.084)	(x0.885)			2.284	(x0.187)	(x7.21)		
							22.02	(x0.042)	(x0.022)	(x0.047)	(x0.026)	182	(x0.051)	(x0.04)	(x0.058)	(x0.046)
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.079	(x0.076)	(x0.591)			2.226	(x0.104)	(x2.08)		
							18.18	(x0.027)	(x0.011)	(x0.028)	(x0.012)	73.61	(x0.037)	(x0.023)	(x0.042)	(x0.03)
8	$g\bar{g} \rightarrow g\bar{g} + (n-2)g$	N/A					2.217	(x0.09)	(x1.78)			3.041	(x0.389)	(x24.9)		
							53.49	(x0.031)	(x0.021)	(x0.04)	(x0.031)	1.24e+03	(x0.026)	(x0.021)	(x0.03)	(x0.027)
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.408	(x0.108)	(x2.46)		
												261	(x0.008)	(x0.006)	(x0.011)	(x0.009)
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		1.87	1.91	1.89	1.89		2.02	2.22	2.07	2.06		1.97	3.04	2.22	2.06	
		x0.069	x0.069	x0.069	x0.069	x0.069	x0.073	x0.09	x0.079	x0.076	x0.079	x0.075	x0.389	x0.136	x0.104	x0.149
summary: run		10.1	11	10.5	10.5		15.2	53.5	27.1	23.1		31.1	1.24e3	234	88.9	
		x0.016	x0.017	x0.017	x0.017	x0.017	x0.011	x0.042	x0.021	x0.015	x0.021	x0.008	x0.051	x0.02	x0.014	x0.024

Generic NLC Performance Matrix (continued)

ID	base process	n=4				n=5				n=6	
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.377	(x0.307)	(N/A)		4.711	(x1.93)	(N/A)		N/A	
		972	(x0.018) (x0.015)	(N/A)		2.26e+04	(x0.023) (x0.022)	(N/A)			
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.26	(x0.229)	(N/A)		3.591	(x1.67)	(N/A)		N/A	
		624	(x0.018) (x0.014)	(N/A)		1.47e+04	(x0.022) (x0.02)	(N/A)			
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.106	(x0.096)	(N/A)		3.128	(x0.225)	(N/A)		N/A	
		159	(x0.009) (x0.007)	(N/A)		1.83e+03	(x0.008) (x0.006)	(N/A)			
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.021	(x0.082)	(N/A)		2.652	(x0.155)	(N/A)		N/A	
		101	(x0.008) (x0.005)	(N/A)		966	(x0.007) (x0.006)	(N/A)			
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	2.233	(x0.188)	(N/A)		4.119	(x0.63)	(N/A)		N/A	
		769	(x0.009) (x0.007)	(N/A)		1.16e+04	(x0.008) (x0.007)	(N/A)			
6	$gg \rightarrow t\bar{t} + (n-2)g$	4.118	(x1.17)	(N/A)		26.71	(x8.36)	(N/A)		N/A	
		4.41e+03	(x0.071) (x0.063)	(N/A)		1.38e+05	(x0.173) (x0.17)	(N/A)			
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	3.683	(x0.303)	(N/A)		16.12	(x1.20)	(N/A)		N/A	
		927	(x0.04) (x0.034)	(N/A)		2.35e+04	(x0.072) (x0.068)	(N/A)			
8	$gg \rightarrow gg + (n-2)g$	12.99	(x2.00)	(N/A)		160	(x11.1)	(N/A)		N/A	
		4.24e+04	(x0.039) (x0.036)	(N/A)		2.05e+06	(x0.073) (x0.072)	(N/A)			
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	2.98	(x0.153)	(N/A)		6.688	(x0.275)	(N/A)		N/A	
		1.25e+03	(x0.005) (x0.004)	(N/A)		1.2e+04	(x0.003) (x0.003)	(N/A)			
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	3.867	(x0.058)	(N/A)		4.664	(x0.063)	(N/A)		N/A	
		102	(x0.016) (x0.012)	(N/A)		358	(x0.008) (x0.007)	(N/A)			
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	2.676	(x0.119)	(N/A)		4.206	(x0.295)	(N/A)		N/A	
		189	(x0.021) (x0.017)	(N/A)		1.81e+03	(x0.015) (x0.014)	(N/A)			
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	4.034	(x0.055)	(N/A)		5.128	(x0.056)	(N/A)		N/A	
		220	(x0.007) (x0.005)	(N/A)		828	(x0.003) (x0.003)	(N/A)			
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	0.27	N/A		N/A	1.385	N/A		N/A	
		N/A	(3.4)	N/A		N/A	(51.7)	N/A			
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A				N/A				N/A	
summary: gen		2.02	13	3.78	2.83	2.65	160	20.1	4.69	N/A	
		x0.055	x2.00	x0.396	x0.17 x0.775	x0.056	x11.1	x2.16	x0.463 x8.42	N/A	
summary: run		101	4.24e4	4.35e3	697	358	2.05e6	1.9e5	1.18e4	N/A	
		x0.005	x0.071	x0.022	x0.017 x0.04	x0.003	x0.173	x0.035	x0.012 x0.077	N/A	

Each cell is raw-library AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. Entries are generation seconds above runtime microseconds per point; PYAMPLICOL runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AMPLICOL reference. Green/orange/red means below one/below two/at least two. Summaries are min|max|avg|med|sum; sum compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8} .

Settings. docs/result_matrix.py (-color-accuracy nlc) writes this table and its JSON cache. AMPLICOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. Only C++ O3 generation is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed PYAMPLICOL cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 24 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AMPLICOL/JIT data remain shown.

10 Generic Full-Colour Performance Matrix

Cell format: generation time above runtime per phase-space point; slots are AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. AMPLICOL uses raw generated-library colour probes. Conventions and gaps are summarized after the table.

ID	base process	n=1					n=2					n=3				
1	$d\bar{d} \rightarrow Z + (n-1)g$	1.885	(x0.071)	(N/A)			2.061	(x0.076)	(N/A)			2.038	(x0.103)	(N/A)		
		10.75	(x0.018)	(x0.005)	(N/A)		26.31	(x0.015)	(x0.007)	(N/A)		88.47	(x0.019)	(x0.012)	(N/A)	
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	1.908	(x0.071)	(N/A)			2.023	(x0.075)	(N/A)			1.987	(x0.09)	(N/A)		
		10.13	(x0.016)	(x0.004)	(N/A)		23.57	(x0.013)	(x0.006)	(N/A)		68.26	(x0.015)	(x0.01)	(N/A)	
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A					2.054	(x0.073)	(N/A)			2.017	(x0.078)	(N/A)		
							16.08	(x0.016)	(x0.007)	(N/A)		38.14	(x0.011)	(x0.006)	(N/A)	
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A					2.026	(x0.073)	(N/A)			1.976	(x0.074)	(N/A)		
							15.18	(x0.013)	(x0.005)	(N/A)		32.16	(x0.009)	(x0.004)	(N/A)	
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A					2.098	(x0.076)	(N/A)			2.058	(x0.092)	(N/A)		
							39.52	(x0.012)	(x0.006)	(N/A)		116	(x0.009)	(x0.006)	(N/A)	
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A					2.073	(x0.083)	(N/A)			2.275	(x0.187)	(N/A)		
							22.03	(x0.042)	(x0.022)	(N/A)		184	(x0.055)	(x0.039)	(N/A)	
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A					2.057	(x0.076)	(N/A)			2.222	(x0.104)	(N/A)		
							18.26	(x0.027)	(x0.011)	(N/A)		74.1	(x0.036)	(x0.023)	(N/A)	
8	$gg \rightarrow gg + (n-2)g$	N/A					2.161	(x0.092)	(N/A)			3.03	(x0.415)	(N/A)		
							52.76	(x0.03)	(x0.021)	(N/A)		1.26e+03	(x0.03)	(x0.02)	(N/A)	
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A					N/A					2.415	(x0.109)	(N/A)		
												261	(x0.009)	(x0.006)	(N/A)	
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A					N/A					N/A				
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A					N/A					N/A				
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A					N/A					N/A				
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A					N/A					N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		1.89	1.91	1.9	1.9		2.02	2.16	2.07	2.06		1.98	3.03	2.22	2.06	
		x0.071	x0.071	x0.071	x0.071	x0.071	x0.073	x0.092	x0.078	x0.076	x0.078	x0.074	x0.415	x0.139	x0.103	x0.153
summary: run		10.1	10.7	10.4	10.4		15.2	52.8	26.7	22.8		32.2	1.26e3	236	88.5	
		x0.016	x0.018	x0.017	x0.017	x0.017	x0.012	x0.042	x0.021	x0.015	x0.022	x0.009	x0.055	x0.021	x0.015	x0.027

Generic Full-Colour Performance Matrix (continued)

ID	base process	n=4					n=5					n=6				
1	$d\bar{d} \rightarrow Z + (n-1)g$	2.454	(x0.304)	(N/A)			4.865	(x1.96)	(N/A)			N/A				
		975	(x0.02)	(x0.015)	(N/A)		2.33e+04	(x0.026)	(x0.022)	(N/A)						
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	2.271	(x0.235)	(N/A)			3.687	(x1.68)	(N/A)			N/A				
		630	(x0.018)	(x0.014)	(N/A)		1.52e+04	(x0.023)	(x0.02)	(N/A)						
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	2.124	(x0.097)	(N/A)			3.188	(x0.227)	(N/A)			N/A				
		160	(x0.009)	(x0.007)	(N/A)		1.92e+03	(x0.008)	(x0.006)	(N/A)						
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	2.045	(x0.082)	(N/A)			2.745	(x0.157)	(N/A)			N/A				
		103	(x0.008)	(x0.005)	(N/A)		1.01e+03	(x0.007)	(x0.006)	(N/A)						
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	2.225	(x0.192)	(N/A)			4.233	(x0.631)	(N/A)			N/A				
		787	(x0.009)	(x0.006)	(N/A)		1.18e+04	(x0.008)	(x0.007)	(N/A)						
6	$gg \rightarrow t\bar{t} + (n-2)g$	4.145	(x1.17)	(N/A)			27.28	(x8.64)	(N/A)			N/A				
		4.44e+03	(x0.078)	(x0.062)	(N/A)		1.46e+05	(x0.175)	(x0.159)	(N/A)						
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	3.687	(x0.302)	(N/A)			16.14	(x1.21)	(N/A)			N/A				
		941	(x0.043)	(x0.033)	(N/A)		2.42e+04	(x0.071)	(x0.063)	(N/A)						
8	$gg \rightarrow gg + (n-2)g$	13	(x2.33)	(N/A)			157	(x13.2)	(N/A)			N/A				
		4.35e+04	(x0.041)	(x0.035)	(N/A)		2.03e+06	(x0.082)	(x0.075)	(N/A)						
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	3	(x0.149)	(N/A)			6.811	(x0.286)	(N/A)			N/A				
		1.24e+03	(x0.005)	(x0.004)	(N/A)		1.24e+04	(x0.003)	(x0.003)	(N/A)						
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	3.992	(x0.056)	(N/A)			4.768	(x0.063)	(N/A)			N/A				
		103	(x0.016)	(x0.012)	(N/A)		366	(x0.008)	(x0.007)	(N/A)						
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	2.668	(x0.12)	(N/A)			4.311	(x0.292)	(N/A)			N/A				
		188	(x0.022)	(x0.016)	(N/A)		1.87e+03	(x0.015)	(x0.013)	(N/A)						
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	4.003	(x0.055)	(N/A)			5.236	(x0.056)	(N/A)			N/A				
		222	(x0.007)	(x0.005)	(N/A)		844	(x0.003)	(x0.003)	(N/A)						
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	0.269	(N/A)			N/A	1.413	(N/A)			N/A				
		N/A	(3.46)	(N/A)			N/A	(55.5)	(N/A)			N/A				
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A					N/A					N/A				
summary: gen		2.04	13	3.8	2.83		2.75	157	20	4.82		N/A				
		x0.055	x2.33	x0.424	x0.17	x0.866	x0.056	x13.2	x2.37	x0.462	x9.81	N/A				
summary: run		103	4.35e4	4.44e3	709		366	2.03e6	1.9e5	1.21e4		N/A				
		x0.005	x0.078	x0.023	x0.017	x0.042	x0.003	x0.175	x0.036	x0.012	x0.086	N/A				

Each cell is raw-library AMPLICOL, PYAMPLICOL JIT O1, and PYAMPLICOL C++ O3. Entries are generation seconds above runtime microseconds per point; PYAMPLICOL runtime shows coloured wall and black pure-evaluator ratios, or absolute time without an AMPLICOL reference. Green/orange/red means below one/below two/at least two. Summaries are min|max|avg|med|sum; sum compares paired totals. A **VALIDATION FAILED** marker means the same-point difference exceeds 10^{-8} .

Settings. docs/result_matrix.py (-color-accuracy full) writes this table and its JSON cache. AMPLICOL uses the raw generated-library path `amplicol_generate -library=create-raw` followed by `amplicol_color_library_probe`, preserving NLC/full colour-order interferences. PYAMPLICOL uses Rusticol double precision, batch size 64, output chunk size 128, Stage-local inputs, ten Horner iterations, default CPE rounds, expanded optimization limits, SymJIT O1, and `-target-runtime 10` are used. Only C++ O3 generation is subject to the 15-minute compile cap.

Fortran reference gaps. 2 completed PYAMPLICOL cells have no direct Fortran colour reference because that path stops above two quark lines; they are shown as absolute timings.

C++ O3 gaps. 43 optional cells were not run and 0 exceeded the 15-minute compile budget; completed AMPLICOL/JIT data remain shown.

Archived low-multiplicity validation fixture. The $n \leq 4$ LC/NLC/full-colour validation cells are mirrored in `tests/fixtures/result_matrix_references.json`. The integration test checks those cached same-point `AMPLICOL`/`PYAMPLICOL` values without a local Fortran run; `scripts/refresh_result_matrix_fixtures.py` refreshes the fixture from the matrix caches.

11 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against `AMPLICOL`. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

Python staged route: source fill $\rightarrow$ bounded stage evaluators $\rightarrow$ amplitude/raw-sum evaluator,
Rusticol route: one PyO3 call $\rightarrow$ Rust source fill and staged sweep $\rightarrow$ amplitude/raw-sum evaluator.

Thus `PYAMPLICOL` keeps `AMPLICOL`'s essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spenso/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The old fully monolithic compiled-DAG route is now deprecated and kept only as reference code. Its alias-based design was useful for exploring whole-graph compilation, but the production target is the staged DAG: Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators. This keeps the runtime schedule close to `AMPLICOL`'s current-table library while leaving the graph compiler model-driven and process-generic.